

GSoC 2026 — Canonical Master Release Track (CMRT) based on the Complete Chromaprint Fingerprint

Organization: Mixxx

Applicant: Swarnadip Kar

Institution: IIT Bhilai, B.Tech Computer Science (3rd Year)

GitHub: [Swarnadip-Kar](#)

Timezone: IST (UTC+5:30)

Mentors: Evelynne, Daniel Schürmann

About Me

I am a third-year CS student at IIT Bhilai. I have been playing keyboard for seven years and use Ableton Live and Logic Pro for live looping performances. I built a touch-based UI called [TouchLooper](#) that sends OSC messages from my phone to control Ableton during live gigs, which got me curious about how DJ software works under the hood. Finding that Mixxx is open source and written in C++/Qt, I wanted to understand it properly.

On the systems side, my OS coursework involved working directly against the [xv6 kernel in C](#) — adding system calls end-to-end, implementing a priority-based scheduler, and solving classical synchronization problems with pthreads and semaphores. I also built a [course compiler](#) covering the full pipeline from lexer through code generation using Flex and Bison.

I joined the Mixxx community in February 2026. Since then I have had eight PRs merged:

- [#15972](#) — Fix silent behaviour when rescanning with no library folders
- [#16066](#) — Key Comparison builtin effect (piano synthesizer reference tone)
- [#16025](#) — BPM lock toggle in DlgTrackInfo
- [#16026](#) — 5/4 and 4/5 BPM scaling controls
- [#16029](#) — Editable tuning field in DlgTrackInfo
- [#16127](#) — Chord mode for the Key Comparison effect
- [#16141](#) — Show resulting BPM on scale buttons
- [#16142](#) — BPM tab layout improvements

A "No Splitting" option for lossless recording formats ([#16132](#)) is currently in review.

I also wrote a Python benchmarking script for the Harmonix Set that connects to mixxxdb.sqlite, evaluates beat grid accuracy using the F-measure metric, and generates interactive HTML visualizations. That work led me deeper into the fingerprint identification problem and eventually to CMRT.

The Problem

DJ-Intelligence: Why This Project Exists

This project forms the base for a bigger vision we call **DJ-Intelligence (DJ-I)** — the idea of sharing the next level of metadata: data and information that particularly interests DJs and music professionals. Things like POIs (Points

of Interest) such as BPM shift points, audio start/end, cue points, key shift points, beat grids, downbeats, genres, ratings, and so on.

Currently every DJ software ships its own analyzers and its own interpretation of the results. There is no single "source of truth" to give a DJ correct BPM values (including different segments within a track) or correct musical keys (including key changes). If we want to share and exchange this kind of advanced metadata between DJs — through MusicBrainz/ListenBrainz for instance — we first need to solve a fundamental question: **are we actually talking about the same audio?**

Together with the companion GSoC idea "Graphical representation of BPM and Musical Keys in the waveform", this project lays the groundwork for DJ-I. The graphical project will need to save BPM curves, key curves, and POI data in an interchangeable format — and that data must be perfectly linked to a CMRT so it can be recreated on other systems and other files of the same mastering.

Recording vs. Mastering: The Core Distinction

MusicBrainz defines a **recording** as a unique audio event — a particular performance captured in a studio or live. A single recording (identified by its MBID — MusicBrainz Identifier) can have many different **masterings**: the version on the original album, the 7-inch single cut, the "best of" compilation version, the streaming-optimized version, the music video version, the Various Artists compilation master, and so on.

A recording is not equal to a mastering. Each mastering may differ in loudness, EQ, dynamic range, intro/outro length, or subtle timing shifts introduced during the mastering process. For a DJ, these differences matter — a beat grid computed on the original album version of a track will not line up perfectly with the "best of" remaster, even though both are the "same song."

How AcoustID Currently Works (and Where It Falls Short)

The current identification chain works like this:

1. The `fpcalc` utility (or Mixxx's built-in `ChromaPrinter`) converts audio to **mono at 11,025 Hz** and computes a Chromaprint fingerprint on only the **first 120 seconds** of audio. This is the same conversion that Picard (MusicBrainz's tagging tool) performs. The hard-coded limit lives in `src/musicbrainz/chromaprinter.cpp` as `constexpr SINT kFingerprintDuration = 120`.
2. This truncated fingerprint is submitted to AcoustID **along with metadata** — title, artist, album, duration. The AcoustID service searches its database for a match and returns an identifier with a confidence score (0.0 to 1.0). AcoustID returns a **confidence percentage rather than a definitive match**.
3. When a fingerprint is submitted, **the AcoustID server combines fingerprints that are similar enough and assigns them a single AcoustID**. AcoustID entries are linked to MBIDs automatically when a user submits a fingerprint from an already-tagged file; MusicBrainz contributors can also manually correct or re-link entries when needed. The result is that multiple AcoustID entries — each representing a different mastering of the same song — can end up pointing to the same MBID, because the MBID represents the *recording* (the audio event), not the specific mastering.
4. This is a consequence of how MusicBrainz models music: a recording is a unique audio event, and all its masterings share one MBID. The data model was not designed with mastering-level distinction in mind. The proposed fix is mastering subclustering — creating finer-grained groupings *within* an existing MBID cluster. That work lives on the MusicBrainz side and is a separate project.

The AcoustID fingerprint comparison tool demonstrates this concretely with Bohemian Rhapsody. The track has fingerprints scattered across multiple AcoustID entries (one with **72 submitted fingerprints**, another with **33**, another with **19**, etc.), and the linked MBIDs are mixed together. The AcoustID compare tool shows offsets between fingerprints within the same cluster (for example, **offset = 10 items** between two submissions, which is roughly 1.24 seconds of audio). This confirms that different masterings do end up lumped together and that there are measurable timing differences between them.

The solution is to use the *complete* Chromaprint fingerprint — the full duration, not just the first two minutes — as an identifier for a specific mastering. Two copies of a track with identical complete fingerprints are provably the same mastering. From there the pregap offset between any two masterings can be calculated, and eventually this information can be shared through MusicBrainz so DJs worldwide can retrieve timing metadata that is guaranteed to match their specific version of a track.

What Is a Canonical Master Release Track?

Once we can identify tracks as being the same mastering, we can choose one version as the **Canonical Master Release Track (CMRT)** — the highest quality version of that mastering available. In the scope of this project (a DJ's local collection), the CMRT is simply the highest-quality file for a given mastering. For example, if a DJ has both a 128 kbps MP3 and a FLAC of the same mastering, the FLAC is the local CMRT. The offset of all other tracks of the same mastering needs to be calculated relative to this CMRT.

In a broader scope (across all users contributing to MusicBrainz), the CMRT would be the highest-quality version of each mastering that has been fingerprinted and registered — the data of highest quality available across the collections of all DJs, stored in the MBDB. This larger-scope CMRT subclustering is the separate MusicBrainz project mentioned above.

When we detect mastering duplicates in a DJ's collection, we suggest the DJ keep the CMRT (highest quality) and optionally remove the lower-quality duplicates.

What I Will Build

1. New database table for fingerprint data

I will add a `track_fingerprints` table to `mixxxdb.sqlite` by adding a new revision to `res/schema.xml` (currently at version 40, so this would be version 41) and bumping `kRequiredSchemaVersion` in `src/database/mixxxdb.cpp` from 40 to 41.

The table stores the complete fingerprint alongside the track reference:

```
CREATE TABLE IF NOT EXISTS track_fingerprints (
  track_id      INTEGER NOT NULL REFERENCES library(id) ON DELETE CASCADE,
  fingerprint    BLOB NOT NULL,
  fingerprint_date TEXT NOT NULL,
  mbid          TEXT,
  pregap_offset REAL,
  is_cmrt       INTEGER DEFAULT 0,
  mb_cmrt_id    TEXT,
  PRIMARY KEY (track_id)
);
```

I store the fingerprint as BLOB rather than TEXT because the raw fingerprint is a sequence of uint32 values. Storing them as a binary blob is more compact and avoids base64 encoding overhead when we need to do bitwise comparisons for offset calculation. For display and network transfer we can encode on the fly.

Why `fingerprint_date`: A track file can change — re-encoded, re-tagged with padding changes, replaced by another version — and the fingerprint must be recomputed if the file modification date is newer than the fingerprint date. Without this timestamp we would have no way to know if the stored fingerprint is still accurate.

Why `mbid`: Mixxx already writes the MusicBrainz Recording ID to file tags via the existing MB lookup (the `DlgTagFetcher` / `TagFetcher` pipeline that goes Chromaprint → AcoustID → MBID → MusicBrainz track releases).

We save it here alongside the fingerprint so we can group tracks by recording without re-reading file tags every time.

Why `is_cmrt` : Boolean flag (0 or 1). Marks whether this track is the Canonical Master Release Track for its mastering group in the local collection. At the moment, this is the track with the highest audio quality (lossless over lossy, higher bitrate over lower) for a given mastering. The offset of all other tracks in the same mastering group is calculated relative to this track.

Why `mb_cmrt_id` : Reserved for the future MusicBrainz CMRT identifier. Once the MusicBrainz-side mastering subclustering infrastructure exists, this field will store the identifier linking the local CMRT to its counterpart in the MBDB, enabling cross-user metadata sharing.

The migration will be backwards-compatible (it only adds a new table, does not modify existing ones), so `min_compatible` can be set to the previous version to allow older Mixxx to still open the database.

I will also write a DAO class (`TrackFingerprintDAO`) to handle reads and writes against this table. This follows the pattern used by `AnalysisDao` in `src/library/dao/analysisdao.cpp` — it takes a `QSqlDatabase` reference in its `initialize()` method. The DAO will provide:

- `saveFingerprintForTrack(TrackId, QByteArray fingerprint, QString date, ...)`
- `getFingerprintForTrack(TrackId)` returning the stored fingerprint
- `getTracksWithMatchingFingerprint(QByteArray fingerprint)` for duplicate detection
- `getAllFingerprintedTrackIds()` for batch queries

2. Chromaprint analyzer step

I will add `AnalyzerChromaprint` in `src/analyzer/analyzerchromaprint.h` and `src/analyzer/analyzerchromaprint.cpp`, following the same structure as the existing analyzers (`AnalyzerBeats`, `AnalyzerKey`, `AnalyzerGain`).

The class subclasses `Analyzer` and implements:

`initialize()` : Check if we already have a fingerprint stored for this track by querying `TrackFingerprintDAO`. If yes, return false (skip). Otherwise, create a Chromaprint context with `chromaprint_new(CHROMAPRINT_ALGORITHM_DEFAULT)` and call `chromaprint_start(ctx, sampleRate, channelCount)`. Store the sample rate so we know the time resolution of the fingerprint later.

`processSamples()` : The analyzer pipeline feeds audio in chunks (the chunk size is defined as `kAnalysisSamplesPerChunk` in `src/analyzer/constants.h`). For each chunk, convert from float to int16 using `SampleUtil::convertFloat32ToS16()` (same conversion the existing `chromaprinter.cpp` does) and call `chromaprint_feed(ctx, samples, size)`. The difference from the existing `ChromaPrinter` class is that we process the *entire* track in chunks rather than reading 120 seconds all at once.

`storeResults()` : Call `chromaprint_finish(ctx)`, then `chromaprint_get_raw_fingerprint(ctx, &fprint, &size)` to get the raw uint32 array. Save it as a `QByteArray` blob through `TrackFingerprintDAO`. The raw fingerprint is what we need for offset calculation (bitwise comparison), not the encoded string form.

`cleanup()` : Free the Chromaprint context with `chromaprint_free(ctx)`.

The analyzer gets wired into the pipeline in `AnalyzerThread::doRun()` (in `src/analyzer/analyzerthread.cpp`). Because it needs a database connection for the DAO, it follows the same pattern as `AnalyzerWaveform` which takes a `QSqlDatabase` parameter:

```
// In AnalyzerThread::doRun(), after the existing analyzers:
m_analyzers.push_back(AnalyzerWithState(
    std::make_unique<AnalyzerChromaprint>(m_pConfig, dbConnection)));
```

The database connection is already set up in `doRun()` when `WithWaveform` mode is active. We need to ensure the connection is also available when only fingerprinting is needed. I will check for a new `AnalyzerModeFlags::WithFingerprint` flag or reuse the existing connection logic.

3. Mastering duplicate detection

With complete fingerprints stored locally, finding exact mastering duplicates becomes a straightforward SQL query: group `track_fingerprints` by fingerprint value and return groups with more than one entry.

I will add a "Find Duplicates" action to the library. The implementation queries `TrackFingerprintDAO` for matching fingerprint values and presents the results as a list of track groups. Each group represents files that have identical masterings.

This is useful independently of any MusicBrainz integration. Many DJs accumulate copies of tracks in different formats or from different sources and want to know which files are actually the same audio.

The UI side will be a dialog similar to the existing track properties dialogs I have already worked on in [#16025](#) and [#16029](#). It shows groups of mastering duplicate tracks with their file paths, formats, and bitrates so the user can decide which copy to keep. When mastering duplicates are found, we suggest the DJ keep the CMRT (highest quality).

4. Opt-in preference for MusicBrainz sharing

I will add a checkbox to Preferences > Library (in `src/preferences/dialog/dlgpreflibrary.cpp`) that opts the user in to sharing fingerprint data anonymously with MusicBrainz in a future release. This follows the pattern of the existing "Sync track metadata" checkbox which uses a config key defined in `src/library/library_prefs.h`. The preference will be stored but the actual network submission will be a no-op in this GSoC scope — the local infrastructure that makes sharing possible is what I am delivering.

5. Pregap offset calculation and metadata swapping

Daniel's [PR #15585](#) demonstrates the correlation approach for calculating the offset between two fingerprints. The method works by:

1. Taking the raw fingerprints as uint32 arrays
2. At each candidate offset, counting the number of matching bits between corresponding elements (using XOR and popcount)
3. The offset with the highest bit-match count is the alignment point
4. Each fingerprint item covers approximately 0.1238 seconds (based on the default Chromaprint parameters: 11025 Hz sample rate, 4096-sample FFT with 2/3 overlap), giving sub-beat precision

I will implement this offset calculation building directly on Daniel's working prototype in [PR #15585](#), using Hamming distance comparison across the raw uint32 arrays to find the best alignment offset, and integrate it with the duplicate detection so that the offset between two masterings can be computed and stored in the `pregap_offset` column. I will discuss the implementation approach with Daniel before writing any code.

Once we have the offset, it becomes possible to **swap timing-based information between two tracks that share the same CMRT** — for example, copying a beat grid from one file to another with the calculated offset applied. This is an early, concrete demonstration of the DJ-I metadata interchange concept.

6. If time permits: version upgrade detection

If the core deliverables are complete before the final week, I would like to add the ability to detect when a user has two versions of the same mastering where one is higher quality — for example, a lossy MP3 and a lossless FLAC of the same file. With complete fingerprints stored and pregap offsets calculated, the matching is already done; the remaining work is surfacing it in the library UI so the user can choose which copy to keep.

Guidelines for the companion "Graphical BPM/Key" project

Eve asked me to include guidelines for a future contributor working on the "Graphical representation of BPM and Musical Keys in the waveform" project. The key constraint:

The data to create the graphical representation of BPM curves, musical key curves, and POIs needs to be saved in an interchangeable format for easy recreation on other systems who have a version of the CMRT.

This means:

- All timing positions in the BPM/Key graphical data must be expressed as frame offsets relative to a well-defined anchor point (audio start or first beat).
- When transferring this data to another file of the same mastering, the CMRT offset from the `track_fingerprints` table must be applied to shift all positions accordingly.
- The data format should be self-contained enough that another system (or another CMRT file) can reconstruct the graphical representation without re-analyzing the audio.
- The data must be perfectly linked to the CMRT — this is what makes interchangeability possible.

What is explicitly out of scope

The MusicBrainz-side sub-clustering system, the actual network submission of fingerprints, and the retrieval of advanced metadata through MusicBrainz are all longer-term goals that depend on MusicBrainz infrastructure work. This project builds the Mixxx-side foundation that makes all of that possible.

Where I Will Work in the Codebase

I have read through these areas while working on my existing PRs:

Area	Files	What I will do
Analyzer pipeline	<code>src/analyzer/analyzer.h</code> , <code>src/analyzer/analyzerthread.cpp</code> , <code>src/analyzer/analyzerbeats.h/.cpp</code> , <code>src/analyzer/analyzerkey.h/.cpp</code>	Add <code>AnalyzerChromaprint</code> following the same pattern
Existing Chromaprint	<code>src/musicbrainz/chromaprinter.h/.cpp</code>	Reference for API usage; will not modify (the 120s limit is correct for AcoustID)
AcoustID lookup	<code>src/musicbrainz/web/acoustidlookuptask.cpp</code>	Reference for understanding the current fingerprint flow
Database schema	<code>res/schema.xml</code> , <code>src/database/mixxxdb.cpp</code> , <code>src/database/schemamanager.cpp</code>	Add revision 41, bump <code>kRequiredSchemaVersion</code>
DAO layer	<code>src/library/dao/analysisdao.h/.cpp</code> (as pattern)	Add <code>TrackFingerprintDAO</code>
Library preferences	<code>src/preferences/dialog/dlgpreflibrary.cpp</code> , <code>src/library/library_prefs.h</code>	Add opt-in checkbox
Track properties	<code>src/library/dlgtrackinfo.cpp</code> (already modified in #16025 , #16029)	Potential display of fingerprint status
Daniel's POC	PR #15585 on <code>daschuer:fingerprint_matcher</code> branch	Study and build on the correlation logic

Timeline and Availability

My end-semester exams finish on 8th May. After packing and travel I will be ready from 14th May, which falls within the community bonding period. I have no commitments for the entire summer. College reopens 27th July — I will need a few travel days around then but the harder work is front-loaded. My 4th year has lighter coursework so I can keep contributing after college resumes.

Community bonding (14th May – 24th May)

Study `src/analyzer/analyzerthread.cpp` in depth to understand exactly how analyzers get their audio chunks. Read Daniel's PR #15585 in full and discuss the correlation approach with mentors. Finalize the `track_fingerprints` schema design. Set up a test collection with different masterings of the same track for manual verification.

Week 1–2 (May 25 – June 7)

Write the schema migration (revision 41 in `res/schema.xml`). Implement `TrackFingerprintDAO` with `save/load/query` methods. Write unit tests for the DAO using Google Test (same framework as existing tests in `src/test/`). Verify the migration works on both fresh installs and upgrades from an existing database.

Week 3–4 (June 8 – June 21)

Implement `AnalyzerChromaprint`. Wire it into `AnalyzerThread::doRun()`. Test that the fingerprint is stable across multiple analysis runs of the same file and that it correctly skips tracks that already have a stored fingerprint. Compare the stored complete fingerprint against `fpcalc` output to verify correctness.

Week 5–6 (June 22 – July 5)

Implement mastering duplicate detection. Write the grouping query against `track_fingerprints`. Build the dialog for presenting results. This is self-contained and can be reviewed and merged independently.

Buffer week (July 6 – July 10)

Free time for unexpected delays, catching up, or deeper testing.

Midterm evaluation deadline: July 10

By midterm: the new table, the analyzer, and mastering duplicate detection should all be in a mergeable state. These three pieces form a coherent and usable feature even without the offset work.

Week 7–8 (July 11 – July 24)

Pregap/offset calculation. Build directly on Daniel's working prototype in PR #15585, using Hamming distance comparison across the raw uint32 arrays to find the best alignment offset. Discuss the implementation approach with Daniel before writing any code.

Week 9–10 (July 21 – August 3)

College reopens July 27 with a few travel days. Complete the offset calculation and add the opt-in preference checkbox to the Library preferences page. Add the ability to swap beat grid info between two tracks sharing the same CMRT. The travel period is a natural buffer if the offset work runs over.

Week 11–12 (August 4 – August 17)

Final testing across macOS and Linux. Address reviewer feedback on earlier PRs. Write documentation for the new table and analyzer. Write contributor guidelines for the companion BPM/Key visualization project.

Final submission: August 17–24

Buffer: The core deliverables (table, analyzer, duplicate detection) are each structured so they can be merged independently. If the offset work takes longer than expected, the preference checkbox is simple and the project still meets both the midterm and final evaluation criteria.

Why This Project

I understand the problem from working through it with Eve. In our DM conversation she walked me through the AcoustID fingerprint comparison tool for Bohemian Rhapsody — showing how different masterings end up lumped together under the same AcoustID entries, and how the visual comparison tool already reveals measurable offsets between them. That made the problem concrete for me rather than abstract.

I ran into a version of this same problem myself when building the Harmonix Set benchmarking script. The Harmonix CSV has AcoustIDs but no raw fingerprints, and my audio files were YouTube downloads. I could not verify whether my audio matched the mastering the annotations were made on. The DTW alignment scores helped filter bad tracks, but that is a workaround. The actual problem is that there is no reliable way to confirm two audio files are the same mastering without the complete Chromaprint.

The key comparison effect I shipped in [PR #16066](#) is something I actually use when practicing keyboard. Having skin in the game makes a difference, and I plan to keep contributing to Mixxx after GSoC.

Links

- GitHub: <https://github.com/Swarnadip-Kar>
- Merged PRs: [#15972](#), [#16066](#), [#16025](#), [#16026](#), [#16029](#), [#16127](#), [#16141](#), [#16142](#)
- Open PRs: [#16132](#)
- Harmonix benchmarking reports and fingerprint CSV: shared in the #gsoc Zulip stream
- TouchLooper: <https://github.com/Swarnadip-Kar/TouchLooper>
- xv6 OS project: <https://github.com/Swarnadip-Kar/xv6-sync-multithreading>
- Compiler implementation: <https://github.com/Swarnadip-Kar/compiler-design-implementation>